

GRAFIKA KOMPUTER

*Rancang Bangun Antarmuka Fisik “Smilie-O-Mat” Untuk Kontrol Grafis Real-Time
Berbasis Arduino Dan Integrasi Api Mastodon*

Dosen Pengampu:

Fajar Septian, S.Pd., S.Kom., M.Kom.



Disusun Oleh :

Farras Iqbal Tawakkal (2407411019)

Muhammad Nur Muhammad (2407411022)

Muhammad Rifky Nuryadi Darmawan (2407411009)

TI 3A

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK NEGERI JAKARTA**

2025

KATA PENGANTAR

Puji syukur senantiasa penulis panjatkan kepada Tuhan Yang Maha Esa yang telah memberikan kemudahan, rahmat, dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan proyek akhir dengan judul “Rancang Bangun Antarmuka Fisik ‘Smilie-O-Mat’ untuk Kontrol Grafis Real-Time Berbasis Arduino dan Integrasi API Mastodon” tepat pada waktunya.

Laporan ini disusun untuk memenuhi tugas mata kuliah Grafika Komputer. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin mengucapkan terima kasih kepada:

1. Fajar Septian, S.Pd., S.Kom., M.Kom. selaku dosen pengampu yang telah memberikan bimbingan dan ilmu yang bermanfaat.
2. Orang tua dan keluarga yang senantiasa memberikan dukungan moral maupun material.
3. Rekan-rekan seperjuangan yang telah memberikan semangat dan saran selama pengerjaan proyek ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna dan masih terdapat kekurangan di dalamnya, baik dari segi penyusunan bahasa maupun aspek teknisnya. Oleh karena itu, penulis sangat mengharapkan kritik serta saran yang membangun dari pembaca untuk penyempurnaan makalah ini di masa mendatang.

Semoga laporan ini dapat memberikan manfaat dan wawasan baru bagi pembaca, khususnya dalam bidang Grafika Komputer.

Depok, 10 Desember 2025

Penulis (Kelompok 5)

DAFTAR ISI

KATA PENGANTAR	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penulisan.....	2
1.4 Manfaat Penulisan.....	2
BAB II LANDASAN TEORI.....	4
2.1 Mikrokontroler Arduino Uno	4
2.2 Komponen Elektronika Input.....	4
2.2.1 Potensiometer (Sensor Analog).....	4
2.2.2 Tactile Push Button (Input Digital).....	5
2.3 Perangkat Lunak Processing	5
2.3.1 Komunikasi Serial (Serial Communication).....	5
2.3.2 Sistem Koordinat dan Resolusi Layar.....	5
2.4 Algoritma dan Logika Pemrograman.....	6
2.4.1 Teknik Smoothing (Signal Averaging).....	6
2.4.2 Linear Mapping (Interpolasi).....	6
2.4.3 Multithreading.....	6
2.5 Antarmuka Pemrograman Aplikasi (API) Mastodon.....	7
BAB III METODOLOGI PERANCANGAN	8
3.1 Model Pengembangan Sistem.....	8
3.2 Analisis Kebutuhan	8
3.3 Perancangan Sistem (Blok Diagram).....	8
3.4 Perancangan Perangkat Keras (Hardware)	9
3.4.1 Skema Rangkaian Elektronika	9
3.4.2 Implementasi Sirkuit.....	9
3.5 Perancangan Perangkat Lunak (<i>Software</i>)	9
3.5.1 Algoritma Mikrokontroler (Arduino).....	9
3.5.2 Algoritma Visualisasi (Processing)	10
3.6 Skenario Pengujian	10

BAB IV IMPLEMENTASI DAN PEMBAHASAN	11
4.1 Implementasi Perangkat Keras (<i>Hardware</i>)	11
4.2 Implementasi Perangkat Lunak (<i>Software</i>)	12
4.2.1 Kode Program Mikrokontroler (Arduino)	12
4.2.2 Kode Program Visualisasi (Processing)	13
4.3 Integrasi API Media Sosial	23
4.4 Hasil Pengujian	23
4.4.1 Pengujian Responsivitas Kontrol	23
4.4.2 Pengujian Fitur Unggah (Upload)	24
4.5 Pembahasan dan Analisis Masalah (<i>Troubleshooting</i>)	25
BAB V PENUTUP	27
5.1 Kesimpulan	27
5.2 Saran	28
DAFTAR PUSTAKA	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi di era Revolusi Industri 4.0 telah mengubah cara manusia berinteraksi dengan mesin. Interaksi yang sebelumnya didominasi oleh perangkat input standar seperti keyboard dan mouse, kini berkembang ke arah Physical Computing atau komputasi fisik. Bidang ini memungkinkan manusia berkomunikasi dengan komputer melalui antarmuka fisik yang lebih intuitif dan alami, menjembatani dunia analog dan digital.

Di sisi lain, media sosial telah menjadi platform utama ekspresi diri. Namun, proses pembuatan konten visual seringkali masih dilakukan secara manual melalui perangkat lunak yang kompleks. Terdapat kebutuhan untuk menciptakan alat yang dapat menggabungkan kreativitas visual, interaksi fisik, dan konektivitas jejaring sosial dalam satu sistem yang terintegrasi.

Berdasarkan permasalahan tersebut, proyek ini bertujuan mengembangkan "Smilie-O-Mat", sebuah sistem kontroler fisik berbasis mikrokontroler Arduino Uno. Sistem ini dirancang untuk mengontrol parameter grafis wajah emoji (alis, mata, mulut, dan warna) secara real-time menggunakan potensiometer, dan mengintegrasikan hasilnya ke media sosial Mastodon melalui pemrosesan gambar otomatis. Proyek ini tidak hanya menerapkan konsep elektronika dasar, tetapi juga teknik pemrograman grafis menggunakan Processing dan komunikasi data serial (Serial Communication), serta pemecahan masalah teknis seperti penanganan noise sinyal dan resolusi layar tinggi (HiDPI).

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

- A. Bagaimana merancang antarmuka fisik menggunakan Arduino untuk mengontrol parameter grafis pada perangkat lunak Processing secara real-time?
- B. Bagaimana mengatasi ketidakstabilan sinyal (jitter) pada sensor analog potensiometer agar pergerakan grafis menjadi halus?
- C. Bagaimana menangani masalah penskalaan grafis pada layar beresolusi tinggi (High DPI) saat melakukan pengambilan gambar (screenshot) otomatis?
- D. Bagaimana mengintegrasikan sistem desktop dengan API (Application Programming Interface) media sosial Mastodon untuk pengunggahan konten secara otomatis?

1.3 Tujuan Penulisan

Tujuan dari pelaksanaan proyek dan penyusunan laporan ini adalah:

- A. Membuat sistem antarmuka fisik yang mampu mengontrol ekspresi wajah digital (alis, mata, mulut, warna) menggunakan input sensor analog.
- B. Menerapkan algoritma smoothing (rata-rata) dan teknik interpolasi (easing) untuk meningkatkan kualitas interaksi pengguna.
- C. Mengimplementasikan komunikasi serial dua arah antara perangkat keras (Arduino) dan perangkat lunak (Processing).
- D. Membangun fitur otomatisasi upload gambar ke Mastodon menggunakan multithreading untuk menjaga responsivitas aplikasi.

1.4 Manfaat Penulisan

A. Manfaat Teoritis

Laporan ini diharapkan dapat menambah khazanah pengetahuan mengenai penerapan komunikasi serial, pemrograman grafis prosedural, dan integrasi API dalam konteks sistem interaktif.

B. Manfaat Praktis

- 1) Bagi Penulis: Meningkatkan pemahaman mendalam tentang Physical Computing, penanganan data sensor, dan pemrograman jaringan.
- 2) Bagi Pembaca/Pengembang Lain: Menjadi referensi teknis dalam mengatasi masalah umum pada proyek Arduino-Processing, seperti noise sensor dan masalah kompatibilitas resolusi layar.

- 3) Bagi Pengguna: Menyediakan alat eksperimental yang menyenangkan untuk mengeksplorasi ekspresi digital melalui kontrol fisik.

BAB II

LANDASAN TEORI

2.1 Mikrokontroler Arduino Uno

Arduino Uno adalah papan mikrokontroler berbasis ATmega328P yang bersifat *open-source*. Papan ini dilengkapi dengan berbagai pin input/output digital dan analog yang memungkinkan pengguna untuk berinteraksi dengan komponen elektronik lainnya.

Dalam proyek "Smilie-O-Mat", Arduino Uno berfungsi sebagai unit pemroses pusat (*Central Processing Unit*) yang bertugas:

1. Akuisisi data yaitu membaca sinyal tegangan analog dari sensor potensiometer melalui pin *Analog-to-Digital Converter* (ADC). ADC pada Arduino memiliki resolusi 10-bit, yang berarti dapat mengubah tegangan 0-5V menjadi nilai integer diskrit antara 0 hingga 1023.
2. Pemrosesan sinyal awal atau melakukan perhitungan rata-rata (*averaging*) untuk menstabilkan pembacaan sensor.
3. Transmisi data atau Mengirimkan data yang telah diproses ke komputer melalui protokol komunikasi serial (*Universal Asynchronous Receiver-Transmitter* / UART) melalui kabel USB.

2.2 Komponen Elektronika Input

2.2.1 Potensiometer (Sensor Analog)

Potensiometer adalah resistor variabel yang berfungsi sebagai pembagi tegangan (*voltage divider*). Komponen ini memiliki tiga terminal: dua terminal terhubung ke sumber tegangan (VCC dan GND) dan satu terminal *wiper* (tengah) yang posisinya dapat diubah secara mekanis dengan memutar knob.

Perubahan posisi *wiper* mengubah nilai resistansi rasio, yang pada gilirannya menghasilkan perubahan tegangan output linear. Dalam sistem ini, empat potensiometer digunakan untuk merepresentasikan empat parameter variabel: kelengkungan alis, ukuran pupil mata, kelengkungan mulut, dan spektrum warna wajah (*Hue*).

2.2.2 Tactile Push Button (Input Digital)

Tactile Push Button adalah saklar momen (*momentary switch*) yang hanya menghubungkan sirkuit saat ditekan. Untuk membaca status tombol secara stabil, diperlukan konfigurasi resistor *pull-up* atau *pull-down* guna menghindari kondisi *floating* (mengambang) di mana tegangan tidak terdefinisi.

Proyek ini memanfaatkan fitur **INPUT_PULLUP** internal pada mikrokontroler ATmega328P. Fitur ini mengaktifkan resistor internal (sekitar 20k-50k Ohm) yang menghubungkan pin input ke tegangan positif (HIGH). Logika pembacaannya menjadi *active-low*:

- **Tidak Ditekan:** Pin bernilai 1 (HIGH).
- **Ditekan:** Pin terhubung ke *Ground* (GND) dan bernilai 0 (LOW).

2.3 Perangkat Lunak Processing

Processing adalah bahasa pemrograman dan lingkungan pengembangan terintegrasi (IDE) yang berbasis Java, yang ditujukan untuk komunitas seni visual dan desain. Processing menyediakan pustaka grafis yang kuat untuk menggambar bentuk geometri 2D/3D (*primitives*) dan memanipulasi warna secara *real-time*.

2.3.1 Komunikasi Serial (Serial Communication)

Komunikasi serial adalah metode pertukaran data antara dua perangkat secara berurutan, satu bit dalam satu waktu. Dalam konteks Arduino dan Processing, komunikasi ini terjadi melalui *virtual COM port*. Data dikirimkan dalam format *string* teks (misalnya: "512,100,20,100,0") yang dipisahkan oleh karakter koma (*delimiter*). Processing kemudian mem-parsing *string* tersebut menjadi tipe data numerik (*float* atau *integer*) untuk digunakan dalam variabel grafis.

2.3.2 Sistem Koordinat dan Resolusi Layar

Processing menggunakan sistem koordinat Kartesius di mana titik (0,0) berada di pojok kiri atas layar. Pada layar modern dengan densitas piksel tinggi (*High DPI* atau *Retina Display*), satu piksel logis dalam kode dapat merepresentasikan dua atau lebih piksel fisik. Fungsi `pixelDensity(2)` digunakan untuk memastikan grafis dirender dengan ketajaman tinggi, namun hal ini memerlukan penyesuaian matematis saat melakukan operasi pengambilan gambar (*screenshot*) agar area yang ditangkap sesuai dengan tampilan visual.

2.4 Algoritma dan Logika Pemrograman

2.4.1 Teknik Smoothing (Signal Averaging)

Sinyal analog dari sensor seringkali mengandung noise atau fluktuasi acak (jitter) akibat interferensi elektromagnetik. Untuk mengatasi hal ini, diterapkan algoritma Smoothing sederhana berbasis rata-rata (mean filter).

Mikrokontroler mengambil N sampel data (misalnya $N = 15$) dalam interval waktu singkat, kemudian menghitung rata-ratanya sebelum dikirimkan.

$$\text{Nilai Stabil} = \frac{\sum_{i=1}^N x_i}{N}$$

Teknik ini secara efektif meredam lonjakan nilai ekstrem dan menghasilkan pergerakan grafis yang lebih tenang.

2.4.2 Linear Mapping (Interpolasi)

Nilai mentah dari sensor Arduino berada pada rentang 0 hingga 1023. Agar dapat digunakan untuk mengontrol posisi elemen grafis pada layar (misalnya posisi X dari 50 hingga 300 piksel), digunakan fungsi pemetaan linear (Linear Interpolation).

Rumus pemetaan nilai x dari rentang $[a, b]$ ke rentang $[c, d]$ adalah:

$$f(x) = c + \frac{(d - c)(x - a)}{b - a}$$

Fungsi ini memungkinkan sinkronisasi presisi antara putaran knob fisik dengan pergerakan objek digital.

2.4.3 Multithreading

Multithreading adalah kemampuan CPU untuk mengeksekusi beberapa proses (*threads*) secara konkuren. Dalam aplikasi berbasis antarmuka pengguna (UI), operasi

jaringan yang berat seperti mengunggah gambar ke internet dapat menyebabkan aplikasi "membeku" (*freeze*) jika dilakukan pada *thread* utama (*main thread*). Oleh karena itu, proses pengunggahan ke Mastodon ditempatkan pada *background thread* terpisah, sehingga animasi wajah pada layar tetap responsif meskipun proses pengiriman data sedang berlangsung.

2.5 Antarmuka Pemrograman Aplikasi (API) Mastodon

Mastodon adalah jejaring sosial terdesentralisasi yang menyediakan API (Application Programming Interface) berbasis REST (Representational State Transfer).

Interaksi dengan API Mastodon dalam proyek ini melibatkan dua tahapan utama:

- A. Upload Media: Mengirimkan file gambar biner menggunakan metode HTTP POST dengan tipe konten multipart/form-data. Server merespons dengan ID media unik.
- B. Post Status: Mengirimkan teks status beserta ID media yang telah diunggah sebelumnya untuk mempublikasikan konten ke linimasa (timeline).

Otentikasi dilakukan menggunakan Bearer Token yang disisipkan pada header permintaan HTTP untuk menjamin keamanan akses.

BAB III

METODOLOGI PERANCANGAN

3.1 Model Pengembangan Sistem

Metodologi yang digunakan dalam pengembangan sistem "Smilie-O-Mat" adalah metode *prototyping*. Pendekatan ini dipilih karena memungkinkan pengembangan interaktif antara perangkat keras dan perangkat lunak secara bertahap.

Tahapan perancangan meliputi:

- A. Analisis Kebutuhan (*Requirement Analysis*).
- B. Perancangan Perangkat Keras (*Hardware Design*).
- C. Perancangan Perangkat Lunak (*Software Design*).
- D. Integrasi Sistem dan Pengujian (*Integration & Testing*).

3.2 Analisis Kebutuhan

Sebelum tahap perancangan, dilakukan identifikasi kebutuhan sistem agar alat dapat berfungsi sesuai tujuan.

- A. Input: Sistem membutuhkan input fisik kontinu untuk parameter wajah (alis, mata, mulut, warna) dan input biner untuk pemicu aksi (tombol simpan).
- B. Proses: Sistem harus mampu membaca sinyal analog, menstabilkan *noise*, mengirimkan data secara serial, dan merender grafis secara *real-time*.
- C. Output: Tampilan visual wajah emoji pada layar komputer dan unggahan otomatis ke platform Mastodon.

3.3 Perancangan Sistem (Blok Diagram)

Secara garis besar, arsitektur sistem "Smilie-O-Mat" dapat digambarkan dalam blok diagram berikut. Data mengalir dari sensor fisik menuju mikrokontroler, kemudian dikirim ke komputer untuk divisualisasikan dan diteruskan ke internet.

Keterangan Blok Diagram:

- A. Blok Input: Terdiri dari 4 Potensiometer dan 1 Tombol.
- B. Blok Proses (Hardware): Arduino Uno yang melakukan konversi ADC (*Analog to Digital Converter*) dan *Smoothing*.
- C. Blok Komunikasi: Kabel USB (Serial Communication) sebagai jembatan data.
- D. Blok Output/Visual: Laptop/PC menjalankan Processing untuk menampilkan UI.

E. Blok Cloud: Server API Mastodon untuk penyimpanan hasil gambar.

3.4 Perancangan Perangkat Keras (Hardware)

3.4.1 Skema Rangkaian Elektronik

Perancangan perangkat keras dilakukan di atas breadboard tanpa memerlukan penyolderan. Berikut adalah rincian koneksi pin pada Arduino Uno:

Tabel Konfigurasi Pin Arduino

Komponen	Jenis Input	Pin Arduino	Fungsi Parameter
Potensiometer 1	Analog (0-5V)	A0	Kelengkungan Alis
Potensiometer 2	Analog (0-5V)	A1	Ukuran Pupil Mata
Potensiometer 3	Analog (0-5V)	A2	Lengkungan Mulut
Potensiometer 4	Analog (0-5V)	A3	Warna Wajah (Hue)
Push Button	Digital	D2	Pemicu Upload

3.4.2 Implementasi Sirkuit

- Potensiometer: Kaki kiri dan kanan masing-masing dihubungkan ke VCC (5V) dan GND. Kaki tengah (*wiper*) dihubungkan ke pin Analog (A0-A3).
- Tombol: Satu kaki dihubungkan ke Pin Digital D2, dan kaki seberangnya ke GND. Tidak digunakan resistor eksternal karena pin D2 dikonfigurasi menggunakan mode INPUT_PULLUP pada sisi *software*, yang mengaktifkan resistor internal mikrokontroler sebesar 20k-50k Ohm.

3.5 Perancangan Perangkat Lunak (Software)

3.5.1 Algoritma Mikrokontroler (Arduino)

Perangkat lunak pada sisi Arduino dirancang dengan logika looping yang berfungsi untuk mengakuisisi data sensor secara terus-menerus.

Untuk mengatasi masalah jitter (nilai sensor yang bergetar), diterapkan algoritma Smoothing rata-rata bergerak sederhana.

Langkah-langkah Algoritma Arduino:

- Inisialisasi komunikasi Serial dengan *baudrate* 9600.
- Mendefinisikan pin A0-A3 sebagai INPUT dan D2 sebagai INPUT_PULLUP.

3. Membaca nilai analog dari 4 potensiometer sebanyak 15 kali iterasi cepat.
4. Menghitung nilai rata-rata dari 15 sampel tersebut.
5. Membaca status tombol (0 jika ditekan, 1 jika dilepas).
6. Memformat data menjadi *string* CSV: "val1,val2,val3,val4,btn".
7. Mengirim *string* tersebut ke port Serial.

3.5.2 Algoritma Visualisasi (Processing)

Aplikasi Processing bertugas menerima data dan menggambar wajah. Logika utamanya dibagi menjadi dua *thread*:

A. Main Thread (UI & Drawing)

1. Membaca data *string* dari Serial Buffer.
2. Memecah (*parsing*) *string* CSV menjadi 5 variabel data (integer/float).
3. Melakukan pemetaan nilai (*mapping*) dari rentang sensor (0-1023) ke rentang koordinat layar (pixel).
4. Menerapkan fungsi *Easing* ($x+ = (target - x) * 0.1$) agar pergerakan slider halus.
5. Menggambar elemen wajah (lingkaran, garis, busur) berdasarkan koordinat yang telah diproses.

B. Background Thread (Network & Upload)

Untuk mencegah aplikasi macet (*freeze*) saat proses pengunggahan, digunakan teknik *multithreading*. Saat tombol ditekan:

1. Aplikasi mengambil tangkapan layar (*screenshot*) pada koordinat wajah.
2. Menyimpan gambar sementara (*upload_temp.png*).
3. Membuka *thread* baru untuk melakukan koneksi HTTP POST ke API Mastodon.
4. Mengirim gambar dan teks status, lalu menerima respon JSON dari server.

3.6 Skenario Pengujian

Pengujian sistem dirancang untuk memverifikasi fungsionalitas alat:

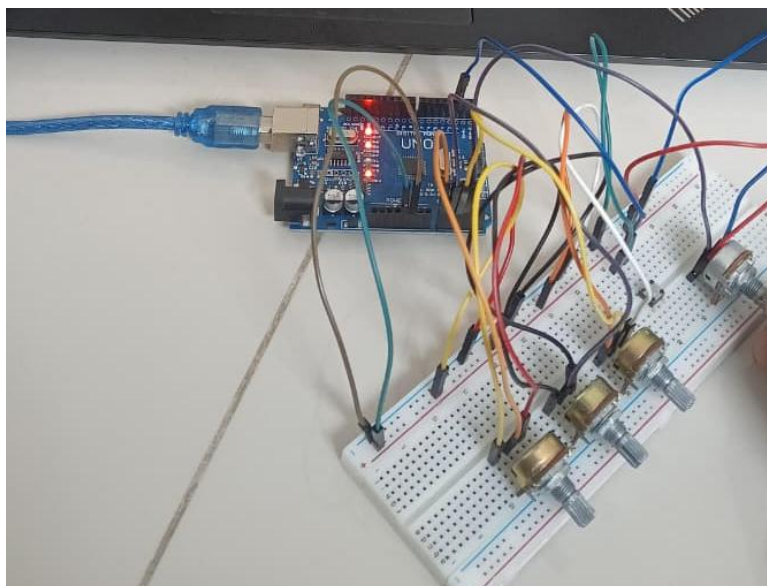
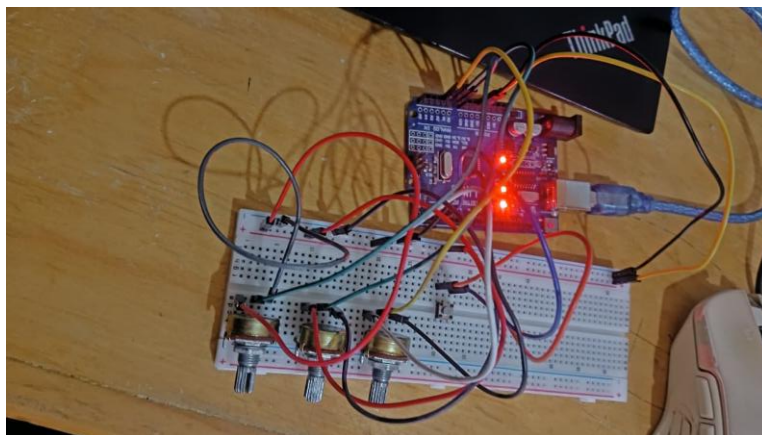
1. Uji Stabilitas Sensor: Memastikan nilai slider tidak bergetar saat knob diam.
2. Uji Responsivitas: Mengukur *delay* antara putaran knob dan pergerakan di layar.
3. Uji Resolusi: Memastikan hasil *screenshot* pada layar resolusi tinggi (HiDPI) tersimpan utuh dan tidak terpotong.
4. Uji Integrasi API: Memastikan gambar berhasil terunggah ke Mastodon dan muncul di *timeline*.

BAB IV IMPLEMENTASI DAN PEMBAHASAN

4.1 Implementasi Perangkat Keras (Hardware)

Tahap implementasi perangkat keras dimulai dengan perakitan komponen pada *breadboard*. Mikrokontroler Arduino Uno dihubungkan ke komputer menggunakan kabel USB tipe B yang berfungsi ganda sebagai suplai daya 5V dan jalur komunikasi data serial.

Rangkaian dibangun sesuai dengan skema yang telah dirancang pada Bab III. Empat buah potensiometer 10k Ohm dipasang secara paralel, di mana seluruh kaki kiri dihubungkan ke *Ground* (GND) dan kaki kanan ke VCC (5V). Output sinyal dari kaki tengah (*wiper*) masing-masing potensiometer dihubungkan ke pin analog A0, A1, A2, dan A3. Sebuah tombol *tactile* dipasang pada pin digital D2 dengan memanfaatkan konfigurasi *input pull-up*.



4.2 Implementasi Perangkat Lunak (*Software*)

4.2.1 Kode Program Mikrokontroler (Arduino)

Pada sisi mikrokontroler, program ditulis menggunakan Arduino IDE. Fokus utama pada tahap ini adalah memastikan pembacaan sensor yang stabil. Untuk mengatasi fluktuasi sinyal (*noise*) yang menyebabkan nilai slider bergetar (*jitter*), diimplementasikan fungsi `bacaRataRata()`. Fungsi ini mengambil sampel data sebanyak 15 kali dalam interval 1 milidetik, kemudian menghitung nilai rata-ratanya sebelum dikirimkan.

Berikut adalah cuplikan implementasi algoritma *smoothing* dan pengiriman data pada Arduino:

```
/*
 * KODE ARDUINO - SMILIE-O-MAT
 */

#define TOMBOL_PIN 2

void setup() {
  Serial.begin(9600);
  pinMode(A0, INPUT); // Knob Alis
  pinMode(A1, INPUT); // Knob Mata
  pinMode(A2, INPUT); // Knob Mulut
  pinMode(A3, INPUT); // Knob Warna
  pinMode(TOMBOL_PIN, INPUT_PULLUP);
}

void loop() {
  // Kita ganti analogRead biasa dengan fungsi bacaRataRata()
  int k1 = bacaRataRata(A0);
  int k2 = bacaRataRata(A1);
  int k3 = bacaRataRata(A2);
  int k4 = bacaRataRata(A3);

  int btn = (digitalRead(TOMBOL_PIN) == LOW) ? 1 : 0;

  // Kirim data
  Serial.print(k1); Serial.print(",");
  Serial.print(k2); Serial.print(",");
  Serial.print(k3); Serial.print(",");
  Serial.print(k4); Serial.print(",");
  Serial.println(btn);

  delay(20);
}
```

```
// --- FUNGSI RAHASIA PENGHALUS ANGKA ---
int bacaRataRata(int pin) {
    long total = 0;
    // Baca 15 kali berturut-turut
    for (int i = 0; i < 15; i++) {
        total = total + analogRead(pin);
        delay(1); // Jeda super singkat 1ms
    }
    // Ambil rata-ratanya (Total dibagi 15)
    return total / 15;
}
```

4.2.2 Kode Program Visualisasi (Processing)

Pada sisi komputer, implementasi dilakukan menggunakan Processing 4. Tantangan utama pada tahap ini adalah menangani penerimaan data serial yang cepat dan menggambar grafis beresolusi tinggi.

Untuk menangani perbedaan logika putaran potensiometer (arah putar terbalik), dilakukan penyesuaian pada fungsi pemetaan (map). Nilai input 0-1023 dipetakan secara terbalik menjadi koordinat layar sliderEndX ke sliderStartX pada variabel yang membutuhkan. Selain itu, teknik interpolasi (*easing*) diterapkan agar pergerakan elemen wajah terlihat mulus dan tidak kaku.

Berikut adalah cuplikan logika penerimaan data (*parsing*) dan pemetaan nilai pada Processing:

```
/**
 * SMILIE-O-MAT: 4 KNOB EDITION
 */

import processing.serial.*;
import java.io.File;
import java.io.OutputStream;
import java.io.PrintWriter;
import java.io.OutputStreamWriter;
```

```

import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.BufferedReader;
import java.net.URL;
import java.net.HttpURLConnection;
import java.net.URLEncoder;
import java.nio.file.Files;
// =====

// 🌀 ISI DATA MASTODON KAMU DI SINI 🌀

// =====

String MASTODON_INSTANCE = "https://mastodon.social";
String ACCESS_TOKEN      =
"CnrDoMaBt8y89atCO_KRJ0b_ZygzFYLM42Dx7BC7lac";
// =====

processing.serial.Serial myPort;

// Variabel Slider

float sliderStartX = 50;
float sliderEndX   = 300;

// Target & Current untuk 4 Knob
float targetEyebrowX = 175, currentEyebrowX = 175;
float targetEyeX     = 175, currentEyeX     = 175;
float targetMouthX   = 175, currentMouthX   = 175;
float targetColorX   = 175, currentColorX   = 175; // (BARU) Knob Warna
float easing          = 0.1;

// Variabel Status
String statusMessage = "Ready";

```

```

int messageTimer = 0;
boolean isPosting = false;
boolean saveRequest = false;
void setup() {
    size(400, 600); // (BARU) Ukuran lebih tinggi biar muat
    pixelDensity(2);
    rectMode(CENTER);
    ellipseMode(CENTER);
    colorMode(HSB, 360, 100, 100);
    println("List Port:");
    printArray(processing.serial.Serial.list());

    // GANTI SESUAI PORT ARDUINO KAMU
    String portName = "COM11";

    try {
        myPort = new processing.serial.Serial(this, portName, 9600);
        myPort.bufferUntil('\n');
    } catch (Exception e) {
        println("⚠️ Gagal konek ke Arduino. Cek Port!");
        statusMessage = "No Arduino";
    }
}

void draw() {
    background(0, 0, 100); // Putih di HSB (Hue, Sat, Brightness=100)
    // --- EASING ---
    currentEyebrowX += (targetEyebrowX - currentEyebrowX) * easing;
    currentEyeX      += (targetEyeX - currentEyeX) * easing;
    currentMouthX    += (targetMouthX - currentMouthX) * easing;
    currentColorX    += (targetColorX - currentColorX) * easing;
    // --- 1. PANEL KONTROL (DIPERBESAR) ---
    fill(0, 0, 92); // Abu-abu muda

```

```

noStroke();
rect(width/2, 500, 400, 200); // Panel lebih tinggi

// --- 2. WAJAH (DENGAN WARNA DINAMIS) ---
float faceHue = map(currentColorX, sliderStartX, sliderEndX, 0, 360);
fill(faceHue, 80, 100); // Warna Wajah Berubah!
stroke(0); strokeWeight(4);
ellipse(width/2, 220, 320, 320);
fill(0, 0, 100); strokeWeight(3);
ellipse(155, 220, 80, 80);
ellipse(245, 220, 80, 80);

// Pupil
float pupilSize = map(currentEyeX, sliderStartX, sliderEndX, 15, 50);
noStroke(); fill(0);
ellipse(155, 220, pupilSize, pupilSize);
ellipse(245, 220, pupilSize, pupilSize);

// Alis
float eyebrowShape = map(currentEyebrowX, sliderStartX, sliderEndX,
15, -15);
stroke(0); strokeWeight(8);
float browY = 160;
line(130, browY, 180, browY - eyebrowShape);
line(220, browY - eyebrowShape, 270, browY);

// Mulut
float mouthBaseY = 310;
float mouthVal = map(currentMouthX, sliderStartX, sliderEndX, -1, 1);
noFill(); strokeWeight(5);

```

```

if (mouthVal > 0.1) {
    float h = map(mouthVal, 0.1, 1, 0, 90);
    float y = map(mouthVal, 0.1, 1, mouthBaseY, mouthBaseY - 5);
    arc(200, y, 120, h, 0.3, PI - 0.3);

} else if (mouthVal < -0.1) {
    float h = map(mouthVal, -0.1, -1, 0, 90);
    float y = map(mouthVal, -0.1, -1, mouthBaseY, mouthBaseY + 10);
    arc(200, y, 120, h, PI + 0.3, TWO_PI - 0.3);

} else {
    stroke(0);
    line(140, mouthBaseY, 260, mouthBaseY);
}

// --- 3. SLIDER VISUAL (ADA 4 SEKARANG) ---
stroke(0); strokeWeight(4);

float y1 = 440; // Alis
float y2 = 470; // Mata
float y3 = 500; // Mulut
float y4 = 530; // Warna (BARU)

line/sliderStartX, y1, sliderEndX, y1);
line/sliderStartX, y2, sliderEndX, y2);
line/sliderStartX, y3, sliderEndX, y3);
line/sliderStartX, y4, sliderEndX, y4); // Garis ke-4

```

```

fill(0); noStroke();
rect(currentEyebrowX, y1, 20, 20);
rect(currentEyeX, y2, 20, 20);
rect(currentMouthX, y3, 20, 20);

// Knob Warna (Kita kasih warna knobnya biar keren)

fill(faceHue, 100, 100);
stroke(0); strokeWeight(2);
rect(currentColorX, y4, 20, 20);

// --- 4. TOMBOL POST ---

if (isPosting) fill(0, 0, 80); else fill(0, 0, 100);
stroke(0); strokeWeight(3);

// Tombol dipindah agak ke bawah (Y=530) di samping slider warna
rect(350, 530, 80, 60);
fill(0); textAlign(CENTER, CENTER); textSize(20);
if (isPosting) text("...", 350, 530);
else text("Post", 350, 530);

// --- 5. STATUS MESSAGE ---

if (messageTimer > 0 || isPosting) {
  if (statusMessage.contains("Error")) fill(0, 100, 100); // Merah
  else fill(120, 100, 80); // Hijau
  textSize(14);

```

```

    text(statusMessage, 350, 575); // Pindah ke paling bawah
    if (!isPosting) messageTimer--;

}

if (saveRequest) {
    saveRequest = false;
    startPostingProcess();
}

}

// =====
// 🧠 LOGIKA SERIAL (5 DATA)
// =====
void serialEvent(processing.serial.Serial p) {
    String s = p.readStringUntil('\n');
    if (s != null) {
        s = trim(s);
        String[] parts = split(s, ',');

        if (parts.length == 5) {
            float val0 = float(parts[0]);
            float val1 = float(parts[1]);
            float val2 = float(parts[2]);
            float val3 = float(parts[3]); // Knob Warna
            float val4 = float(parts[4]); // Tombol sekarang index ke-4
            targetEyebrowX = map(val0, 0, 1023, sliderEndX, sliderStartX);
            targetEyeX      = map(val1, 0, 1023, sliderEndX, sliderStartX);
            targetMouthX    = map(val2, 0, 1023, sliderEndX, sliderStartX);
            targetColorX    = map(val3, 0, 1023, sliderEndX, sliderStartX); //
Mapping Knob 4

```

```

        if (val4 == 1 && !isPosting) {
            saveRequest = true;
        }
    }
}

// =====
// 📷 LOGIKA SCREENSHOT & POSTING
// =====

void startPostingProcess() {
    isPosting = true;
    statusMessage = "Uploading...";
    PImage shot = get(38*2, 58*2, 324*2, 324*2);
    String filename = "upload_temp.png";
    shot.save(savePath(filename));
    thread("uploadAndPostThread");
}

void uploadAndPostThread() {
    String filePath = savePath("upload_temp.png");
    String mediaId = uploadMediaMastodon(filePath);
    if (mediaId != null) {
        boolean success = postStatusMastodon("Controlling emote by smilie
o-mat controller! #Arduino #Processing, #RifkyIqbalNur", mediaId);
        if (success) statusMessage = "Posted!";
        else statusMessage = "Error Posting";
    } else {
        statusMessage = "Error Upload";
    }
}

```

```

        isPosting = false;
        messageTimer = 180;
    }

String uploadMediaMastodon(String path) {
    // (SAMA SEPERTI SEBELUMNYA, TIDAK BERUBAH)
    try {
        String charset = "UTF-8";
        File binaryFile = new File(path);
        String boundary = Long.toHexString(System.currentTimeMillis());
        String CRLF = "\r\n";
        URL url = new URL(MASTODON_INSTANCE + "/api/v2/media");
        HttpURLConnection connection = (HttpURLConnection)
url.openConnection();
        connection.setDoOutput(true);
        connection.setRequestMethod("POST");
        connection.setRequestProperty("Authorization", "Bearer " +
ACCESS_TOKEN);
        connection.setRequestProperty("Content-Type", "multipart/form-data;
boundary=" + boundary);
        OutputStream output = connection.getOutputStream();
        PrintWriter writer = new PrintWriter(new OutputStreamWriter(output,
charset), true);
        writer.append("--" + boundary).append(CRLF);
        writer.append("Content-Disposition: form-data; name=\"file\";
filename=\"" + binaryFile.getName() + "\"").append(CRLF);
        writer.append("Content-Type: image/png").append(CRLF);
        writer.append(CRLF).flush();
        Files.copy(binaryFile.toPath(), output);
        output.flush();
    }
}

```

```

writer.append(CRLF).flush();
writer.append("--" + boundary + "--").append(CRLF).flush();
int responseCode = connection.getResponseCode();
if (responseCode == 200 || responseCode == 202) {
    InputStream response = connection.getInputStream();
    BufferedReader reader = new BufferedReader(new
InputStreamReader(response));
    StringBuilder json = new StringBuilder();
    String line;
    while((line = reader.readLine()) != null) json.append(line);
    reader.close();
    JSONObject jsonObj = parseJSONObject(json.toString());
    return jsonObj.getString("id");
} else {
    println("Upload Failed. Code: " + responseCode);
}
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

boolean postStatusMastodon(String text, String mediaId) {
    // (SAMA SEPERTI SEBELUMNYA, TIDAK BERUBAH)
    try {
        URL url = new URL(MASTODON_INSTANCE + "/api/v1/statuses");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setDoOutput(true);
        conn.setRequestProperty("Authorization", "Bearer " + ACCESS_TOKEN);
        conn.setRequestProperty("Content-Type", "application/x-www-form-
urlencoded");
    }
}

```

```

    String params = "status=" + URLEncoder.encode(text, "UTF-8") +
"&media_ids[]" + mediaId;
    OutputStream os = conn.getOutputStream();
    os.write(params.getBytes());
    os.flush();
    os.close();
    if (conn.getResponseCode() == 200) return true;
    else println("Post Failed: " + conn.getResponseCode());
} catch (Exception e) {
    e.printStackTrace();
}
return false;
}

```

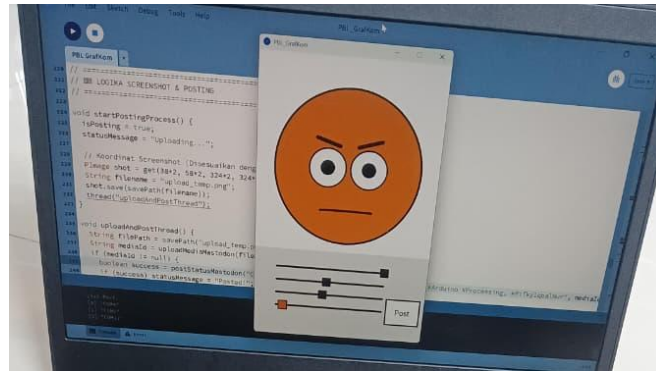
4.3 Integrasi API Media Sosial

Integrasi dengan Mastodon dilakukan menggunakan protokol HTTP POST. Proses ini dijalankan pada *thread* terpisah (*background thread*) agar tidak mengganggu kinerja antarmuka utama saat proses pengunggahan berlangsung. Sistem secara otomatis mengambil tangkapan layar (*screenshot*) area wajah, menyimpannya sementara, lalu mengirimkannya ke *endpoint* API Mastodon.

4.4 Hasil Pengujian

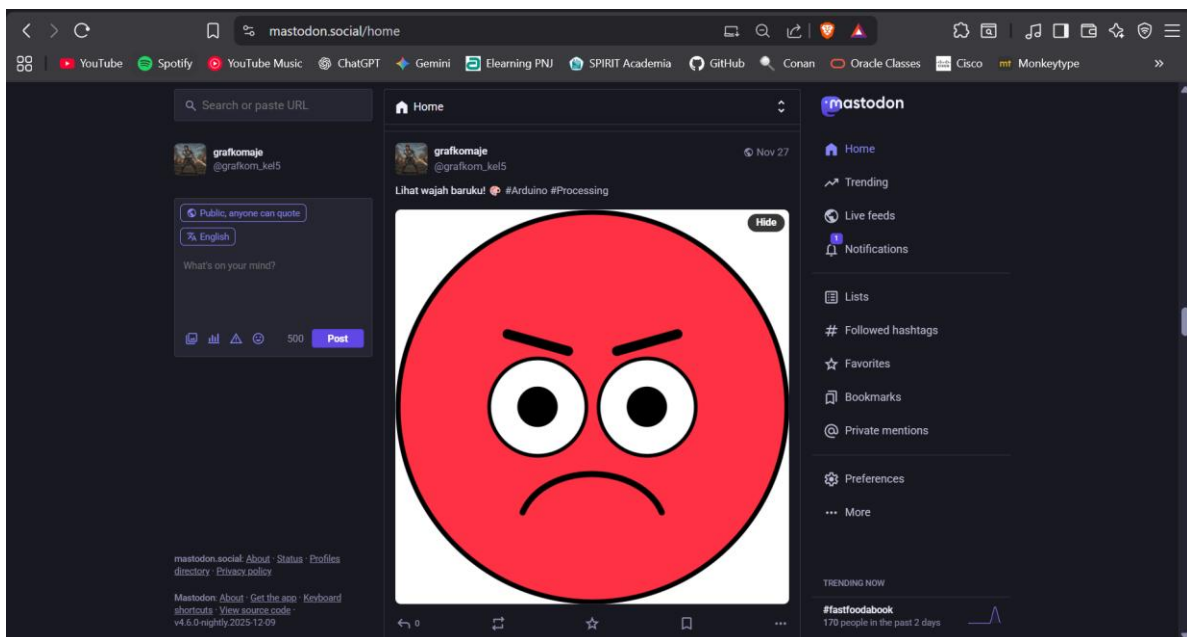
4.4.1 Pengujian Responsivitas Kontrol

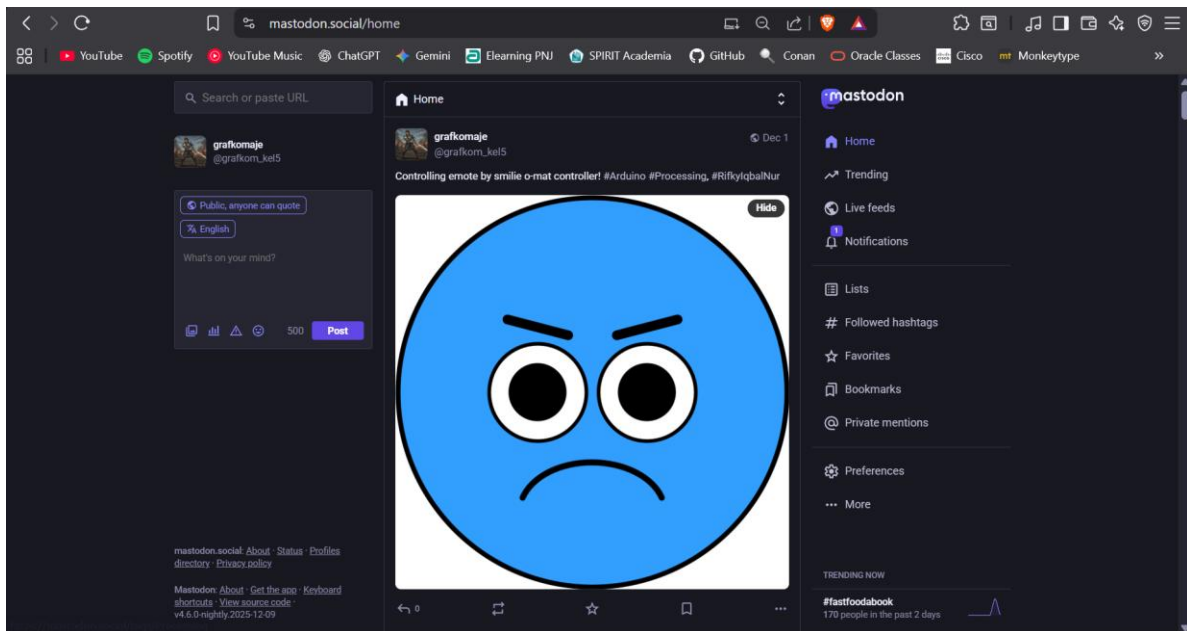
Pengujian dilakukan dengan memutar keempat knob secara acak. Hasil menunjukkan bahwa elemen visual (alis, mata, mulut, dan warna) merespons perubahan posisi knob dengan jeda (*latency*) yang sangat minim. Algoritma *smoothing* terbukti berhasil meredam getaran, sehingga slider pada layar tetap diam saat knob tidak disentuh.



4.4.2 Pengujian Fitur Unggah (Upload)

Pengujian dilakukan dengan menekan tombol fisik pada alat. Sistem berhasil mendeteksi penekanan tombol, mengubah status menjadi "Uploading...", dan dalam waktu kurang dari 5 detik (bergantung kecepatan internet), status berubah menjadi "Posted!". Hasil unggahan pada linimasa Mastodon menunjukkan gambar wajah yang utuh dan presisi, membuktikan bahwa perhitungan koordinat *screenshot* pada layar resolusi tinggi (*High DPI*) telah berfungsi dengan benar.





4.5 Pembahasan dan Analisis Masalah (*Troubleshooting*)

Selama proses pengembangan, ditemukan beberapa kendala teknis yang signifikan. Berikut adalah analisis dan solusi yang diterapkan:

A. Masalah Ambiguitas Pustaka Serial

Saat mengintegrasikan fitur *upload* (yang membutuhkan pustaka `java.io` dan `java.net`) dengan fitur Arduino (pustaka `processing.serial`), terjadi konflik penamaan kelas karena kedua pustaka memiliki kelas bernama `Serial`.

- **Solusi:** Masalah ini diatasi dengan menggunakan referensi kelas secara eksplisit, yaitu menuliskan `processing.serial.Serial` pada saat deklarasi objek, serta mengimpor kelas-kelas Java secara spesifik satu per satu, bukan menggunakan *wildcard* (`import java.io.*`).

B. Isu Tampilan Terpotong pada Layar High-DPI

Pada awal pengujian fitur *screenshot*, gambar yang tersimpan hanya menampilkan sebagian kecil wajah (terbesar/zoom). Hal ini disebabkan oleh fitur `pixelDensity(2)` yang diaktifkan untuk ketajaman layar, namun fungsi `get()` secara *default* mengambil koordinat piksel logis, bukan fisik.

- **Solusi:** Melakukan kalibrasi koordinat pengambilan gambar secara manual. Dengan tetap mengaktifkan `pixelDensity(2)`, Processing secara internal menangani

skala, sehingga koordinat `get()` dikembalikan ke nilai dimensi asli (lebar dan tinggi wajah) tanpa perlu pengalihan manual, menghasilkan gambar yang presisi dan tajam.

C. Fenomena Jitter pada Sensor

Meskipun menggunakan potensiometer berkualitas baik, nilai pembacaan pada Serial Monitor tetap mengalami fluktuasi ± 1 hingga 2 digit. Jika dipetakan langsung ke layar, hal ini menyebabkan efek visual yang "bergetar".

- **Solusi:** Penerapan metode rata-rata bergerak (*moving average*) pada kode Arduino (mengambil 15 sampel) terbukti efektif menghilangkan *noise* tersebut, menghasilkan data keluaran yang stabil dan gerakan animasi yang halus.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem "Smilie-O-Mat", dapat ditarik beberapa kesimpulan sebagai berikut:

A. Keberhasilan Integrasi Hardware-Software

Sistem antarmuka fisik berhasil dibangun menggunakan Arduino Uno dan komponen elektronika dasar. Keempat potensiometer berfungsi efektif mengontrol parameter grafis (alis, mata, mulut, warna) secara independen, dan tombol fisik berfungsi responsif sebagai pemicu aksi.

B. Stabilitas Pembacaan Sensor

Penerapan algoritma *Smoothing* (rata-rata bergerak 15 sampel) pada mikrokontroler terbukti krusial. Algoritma ini berhasil menghilangkan *noise* atau *jitter* pada sinyal analog, sehingga pergerakan elemen visual pada layar menjadi stabil dan tidak bergetar saat *knob* diam.

C. Kualitas Visual dan Skala

Masalah tampilan terpotong (*cropping*) pada layar resolusi tinggi (*HiDPI*) berhasil diatasi tanpa mengorbankan ketajaman gambar. Penggunaan fungsi `pixelDensity(2)` yang dikombinasikan dengan kalibrasi koordinat *screenshot* memungkinkan sistem menghasilkan gambar *output* yang presisi.

D. Kinerja Sistem (Multithreading)

Integrasi dengan API Mastodon berjalan lancar. Penggunaan teknik *multithreading* (pemrosesan latar belakang) berhasil mencegah terjadinya pembekuan (*freeze*) pada antarmuka pengguna saat proses pengunggahan data berlangsung, menjaga pengalaman pengguna tetap mulus (*seamless*).

5.2 Saran

Meskipun sistem telah berjalan sesuai tujuan, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk penelitian atau proyek berikutnya:

A. Pengembangan Nirkabel

Mengganti mikrokontroler Arduino Uno dengan modul berbasis ESP32 atau ESP8266 agar alat dapat terhubung ke komputer atau internet secara nirkabel (*Wi-Fi/Bluetooth*), menghilangkan ketergantungan pada kabel USB.

B. Penambahan Feedback Fisik

Menambahkan komponen *output* pada alat, seperti LED indikator status atau layar LCD kecil untuk menampilkan notifikasi ("Uploading", "Done") secara langsung di perangkat keras tanpa perlu melihat layar komputer.

C. Desain Kemasan (Casing)

Merancang dan mencetak *casing* menggunakan *3D Printer* atau akrilik agar alat terlihat lebih rapi, ergonomis, dan melindungi komponen elektronika dari debu atau sentuhan langsung.

D. Variasi Ekspresi

Mengembangkan algoritma grafis di Processing untuk mendukung lebih banyak variasi bentuk mata atau aksesoris wajah, sehingga ekspresi yang dihasilkan lebih kaya.

DAFTAR PUSTAKA

Arduino. (2024). *Arduino reference: Analog read & input pullup*.

Mastodon. (2024). *Mastodon API documentation: Methods for media uploads and statuses*.

Processing Foundation. (2024). *Processing 4 reference: Serial library & pixel density*.

Reas, C., & Fry, B. (2014). *Processing: A programming handbook for visual designers and artists*. MIT Press.